

pces API

This document describes the formatting of pces model input files, and serves as an API for creating those files. As preparation for this document, one should read [PCES-Introduction.pdf](#), and [PCES-Internals.pdf](#).

Input Files

The interface specification for a pces simulation names 8 files that contain the model specification. From the command line theses are

- `-cp` Names a file that defines the structure of the model's Computational Patterns.
- `-cplnit` Names a file that defines the initial state of model components.
- `-topo` Names a file that defines computer and network topology
- `-map` Names a file which describes the assignment of the application's computational elements to processors.
- `-ipmap` Names a file which describes the mapping of IP addresses on pcap packets presented to the simulator to internal IP addresses, and describes external packet feeds
- `-exp` Names a file that contains a description of assigning performance parameters to network entities
- `-funcExec` Names a file that contains all function execution timing information needed by the model.
- `-devExec` Names a file that contains device operation timing information for routers and switches
- `-experiment` Names a file that describes a set of experiments to run together as a group defining an 'evaluation'

The sections below describe the format of each file, using yaml notion to describe lists, dictionaries, and to some extent data types. However this is not pure yaml, as we embed in this notion model-specific information about keys and values of some of the dictionaries. When a dictionary definition includes a word that is entirely in capital letters, it means that in the actual yaml file one expects a string to be in that location, and that the capitalized word represents a set of strings from which the string must come. In section [Identity Sets](#) we define the set associated with each such word. In addition, the value of a dictionary's key may be given by a keyword that is a mixture of upper and lower case characters. This keyword will be the name of a dictionary structure as it is defined in this document. The dictionary names are types from the go representation of those dictionaries in the pces and mrnes code bodies.

1. `-cp cp.yaml`

The `cp.yaml` file defines the structure of Computational Patterns. The file as a whole is organized as a dictionary with a `dictname` attribute and a dictionary indexed by a `CPTYPE` identifier to associate with a `CompPattern` data structure.

1.1 `CompPatternDict`

```
dictname: string
patterns:
  CPTYPE: CompPattern
```

1.2 CompPattern

A pces computational pattern is basically a graph whose nodes describe computations and whose edges describe flow of messages between the functions that perform the computation. The structure of the CompPattern dictionary is given below.

```
cptype: CPTYPE
name: CPNAME
funcs: [Func]
services:
  string:Func
edges: [CmpPtnGraphEdge]
extedges: [XCPEdge]
```

Here we see that the dictionary has a key 'funcs' whose value is a list of (yet-to-be-defined) Func dictionaries, and likewise the dictionary has a list of CmpPtnGraphEdge dictionaries.

A model may have multiple instances where the structural pattern is the same, and a modeler developing code may wish to leverage that somehow. Correspondingly when we create the dictionary describing a computational pattern we give it both 'cptype' and 'name' attributes, allowing (but not requiring) them to be different.

A CompPattern may have associated with it functions we call 'Services' that a function might cite by name. The services map associates strings that give those functions service names with the functions themselves. The service name given to a function in this way does not need to be the function's 'label' (see below).

A function is permitted to send messages to functions in different computational patterns, in addition to those in its own. We differentiate between intra- and inter- pattern messages with different lists, those found through key 'edges' and those found through key 'extedges'.

1.3 Func

Computational activity is simulated to occur within the context of a Func. The details of how/when a Func's activities take place, the associated simulated execution, the output response, and the state variables the Func uses are all detailed in other places in the model description. To describe computational pattern as a graph we need to specify the identity of the node, through a pair whose key is 'label'. Func behavior is based on its 'class' and so a dictionary pair identifying the class is included here as well. The value associated with the 'class' key must be declared and supported by the pces code base as a recognized class.

```
class: FUNCLASS
label: FUNCLABEL
```

1.4 CmpPtnGraphEdge

In describing a potential information flow between Funcs, we need to identify the Func endpoints. In addition we include pairs that identify the strings that identify message type and method code. The message type here is a label used to differentiate an edge from others with the same source. Receipt of a message delivered to a Func through one of its ingress edges triggers a (simulated) computation and response, and the choice of those is allowed to depend on the identity of the source Func, the message type, and a 'method code' which selects one of the destination Func's possible responses.

The dictionary associated with a CmpPtnGraphEdge is given by

```
srclabel: FUNCNAME
msgtype: MSGTYPE
dstlabel: FUNCNAME
```

Two edges are distinct (and are permitted to co-exist) if they differ in any one of these attributes.

The string mapped to by the 'srclabel' key must be referenced as the target of the 'label' key in some Func in the 'funcs' list of the CompPattern within which the CmpPtnGraphEdge is defined. The same is true of the string mapped to by the 'dstlabel' key.

1.5 XCPEdge

pces permits messages to flow from one CompPattern to another. As the description of a CmpPtnGraphEdge assumes the endpoint functions are co-resident in the same CompPattern, we use a different dictionary to describe external edges. It simply adds attributes naming the computational patterns associated with the source and destination functions, separately.

```
srccp      CPNAME
dstcp      CPNAME
srclabel   FUNCNAME
dstlabel   FUNCNAME
msgtype    MSGLABEL
```

The string mapped to by the 'srclabel' key must be referenced as the target of the 'label' key in some Func in the 'funcs' list of the CompPattern whose 'name' is the target here of the 'srccp' key. An entirely similar requirement exists on the string mapped to by the 'dstlabel' key.

2. -cplnit cplnit.yaml

2.1 CPlnitListDict

The cplnit.yaml file holds data structures used by pces to initialize the run-time representation of the model. Like cp.yaml, the overall file structure holds a dictionary CPlnitListDict, this one being indexed by strings in CPNAME (rather than in CPTYPE) to map to a CPlnitList dictionary.

```
dictname: string
initlist:
  CPNAME: CPOutList
```

2.2 CPOutList

A CPOutList dictionary holds serialized configuration strings for the Funcs of a given instance of a CompPattern, accessible through a dictionary that is indexed by the Func's 'label' attribute. The format of the configuration string is defined by the class of the Func, but the specifics of the configuration depend on the specific identity of the Func. Identity and then format can be recovered by analysis of the 'cfg' dictionary key.

In principle we allow for the configuration string to be a serialized form of JSON, but that has not been much tested. We lean heavily towards YAML through pces/mrnes.

A CPOutList also lists descriptions of all the messages exchanged between those Funcs.

```
name: CPNAME
cptype: CPTYPE
useyaml: bool
cfg:
  FUNCNAME: |
    SERIALCFG
msgs: [CompPatternMsg]
```

2.3 CompPatternMsg

The CPOutList descriptor includes a list of descriptions of messages received and sent by the CompPattern. The description holds the message type label we've seen before, as well as a boolean indicating whether the message is associated with a packet or a flow (the latter being used with multi-resolution traffic modeling).

```
msgtype: MSGTYPE
ispckt: bool
```

3. topo.yaml

File topo.yaml holds information describing the computers and networks in the mrnes model. The API for this file is given in [mrnes-API](#).

4. map.yaml

Every Func of every instance of a CompPattern in a pces model is mapped to some Endpt in the supporting mrnes network; this mapping is described in map.yaml. The file contains a dictionary with a pair assigning a name to the dictionary, and a dictionary 'map' that is indexed by name of computational pattern. The value so referenced is another data structure that includes a dictionary 'funcmap' that is indexed by the value associated with the Func's 'label', with a value being the name of the mrnes endpoint responsible for simulating the

execution of the Func.

```
dictname: string
map:
  CPNAME:
    patternname: CPNAME
    funcmap:
      FUNCLABEL: ENDPTNAME,int
```

Every CompPattern listed in the 'patterns' dictionary of cp.yaml's CompPatternDict dictionary must have a key in this dictionary's 'map' dictionary. Given such a key, say "cpn", the value of 'patternname' is of course "cpn", and its 'funcmap' dictionary must have as keys exactly and only the strings referenced by the 'label' key in all Funcs declared in cpn's CompPattern 'funcs' list. The name of an endpoint associated to an function is concatenated with a comma, and then a positive integer. The integer gives a scheduling priority for executing that function in the presence of contention for CPU resources.

5. ipmap.yaml

A pces/mrnes model may accept inputs from external sources, e.g., pcap packets that were previously recorded or that are being captured on-line. Those packets will contain IP source and destination addresses, but we do not require that the simulation model explicitly use the IP address space from which the packets are derived. ipmap.yaml contains three lists of dictionaries.

```
devices: [deviceDict]
networks: [networkDict]
feeds: [feedDict]
```

Here

```
deviceDict:
  device : ENDPTNAME
  network: NETWORKNAME
  intIP: string
  extIP: string
```

IP addresses are bound to network interfaces, and so we identify an interface by specifying the name of the device it is bound to, and the name of the network it faces. Here we declare that the interface is known within the simulation model by an IP address specified by `intIP`, and if an external packet with source or destination address `extIP` arrives, it is understood that the interface being referenced is the one specified by this dictionary.

The network list maps CIDR blocks to a named network

```
networkDict:
  network: NETWORKNAME
  intCIDR: string
  extCIDR: string
```

This dictionary just states the equivalence of the external network identified by `extCIDR` to be the named network, and gives it an internal CIDR block assignment of `intCIDR`.

One can introduce a feed of packets to the simulator in a number of ways, and the `feedDict` dictionary defines these.

```
feedDict:
  name: string
  active: int
  dilation: float
  srctype: {"file", "unix-socket", "net-socket"}
  srcspec: string
  start: float
  time: {"packet", "clock"}
```

Every feed has name, for reference. We can leave the specifications for a feed in the `ipmap.yaml` file but declare it to be inactive. `active` is an integer encoding of boolean values to specify whether the simulation ought to expect packets to arrive from this feed. Packets can read into the simulator from a file. This involves no inter-process communication, but does require one to specify the source is a file (through the "file" assignment to attribute `srctype`) and the path to the PCAP file to read, given in attribute `srcspec`. The requirement is that the path specified be relative to the directory where the model input files discussed here are declared to be. Packets may be presented to the simulator through sockets also, in which case attribute `srctype` is either "unix-socket" or "net-socket" depending on whether the socket is in the file system shared by both the packet source and the simulator, or through the network. In the former case `srcspec` is the absolute path to the file used to implement the socket, and in the latter case `srcspec` is assigned the port number the simulator will bind to in order to receive packets. The remainder of the `feedDict` dictionary attributes relate to the assignment of virtual time to the arrival of the feed's packets. When the `time` attribute is set to "packet" the simulator will create a virtual time base on the time stamp reported as being derived from the packet, assumed to be the number of micro-seconds in the Unix epoch (expressed in a 64-bit integer). `start` is set to the virtual time to be ascribed to the first packet to be observed in the feed. By saving the 1st packet's native time, on arrival of a subsequent packet one can compute the number of microseconds that elapsed between the dispatch of the first packet and the subsequent one. This difference is scaled by the floating point attribute assigned to `dilation`, which either increase that difference in virtual time coordinates or decrease it, depending on whether `dilation` is great than or less than 1.0.

6. exp.yaml

When a mrnes model is loaded to run, the file `exp.yaml` is read to find performance parameters to assign to network devices, e.g., the speed of a network interface. The API for this file is given in [mrnes-API](#).

7. funcExec.yaml

funcExec.yaml holds descriptions of function timings, dependent on the type of computer on which the execution occurs, and the length of the data packet being processed. A given Func in a CompPattern may perform different computations, depending on the source Func and message type of its input. We therefore call the simulated computations 'operations' and the timings file describes timings of different operations. The FuncExecList dictionary that funcExec.yaml holds contains a dictionary indexed by a string naming the timing, mapping to a description of the timing.

7.1 FuncExecList

```
listname: string
times:
  TIMINGCODE: FuncExecDesc
```

7.2 FuncExecDesc

Description of a timing includes the operation identifier (which is referenced in the executing model code), a modeler-included 'param' attribute which may be used by model code to refine its operations, identity of the model of CPU on which the measurement was taken, the length of the data packet driving the computation, and the measured execution time (in seconds)

```
identifier: string
param: string
cpumodel: CPUMODEL
pktlen: int
exectime: float
```

8. devExec.yaml

devExec.yaml holds a dictionary DevExecList of timings of routers and switches as they route, and switch. The API for this file is given in [mrnes-API](#) .

Identity Sets

The specifications above frequently used a word all in upper case (an "Identity Set") in positions where in conformant yaml one would find the type 'string'. Each identity set represents a set of strings, and the application of the word representing the set is meant to convey the constraint that in actual expression of the dictionary, one of the strings in the identity set is chosen. This does NOT necessarily mean that *any* string in that set might appear there, this technique for expression is not sophisticated enough to convey some dependencies and limitations that exist, but does help to express how declarations in one portion of the model are used in others.

Below we describe each identity set and define the set of strings it represents.

--	--

Identity Set	Composition
CPNAME	All values mapped to by 'name' key in CompPattern dictionaries (1.2)
CPTYPE	All values mapped to by 'cptype' key in CompPattern dictionaries (1.2)
MSGTYPE	All values mapped to by 'msgtype' key in CompPatternMsg dictionaries (1.4)
FUNCLASS	All values mapped to by 'class' key in Func dictionaries (1.3)
FUNCLABEL	All values mapped to by 'label' key in Func dictionaries (1.3)
METHOD	All values mapped to by 'methodcode' key in CompPatternMsg and XCPMsg dictionaries (1.4 and 1.5)
SERIALCFG	Result of serializing a json dictionary or yaml struct of a function's configuration
NETWORKNAME	All values mapped to by 'name' key in NetworkDesc dictionaries (3.2)
ENDPTNAME	All values mapped to by 'name' key in EndptDesc dictionaries (3.3)
CPUMODEL	All values mapped to by 'model' key in EndptDesc dictionaries (3.3)
ROUTERNAME	All values mapped to by 'name' key in RouterDesc dictionaries (3.4)
ROUTERMODEL	All values mapped to by 'model' key in RouterDesc dictionaries (3.4)
SWITCHNAME	All values mapped to by 'name' key in SwitchDesc dictionaries (3.5)
SWITCHMODEL	All values mapped to by 'model' key in SwitchDesc dictionaries (3.5)
INTRFCNAME	All values mapped to by 'name' key in IntrfcDesc dictionaries (3.6)
MEDIA	{"wired", "wireless"}
NETSCALE	{"LAN", "WAN", "T3", "T2", "T1"}
DEVTYPE	{"Endpt", "Router", "Switch"}
DEVNAME	Union of sets ENDPTNAME, ROUTERNAME, SWITCHNAME
GROUPNAME	All values in lists mapped to by key 'groups' in NetworkDesc, EndptDesc, RouterDesc, SwitchDesc, and IntrfcDest dictionaries (3.2, 3.3, 3.4, 3.5, 3.6)
NETWORKOBJ	{"Network", "Endpt", "Router", "Switch", "Interface"}
ATTRIB	{"name", "group", "device", "scale", "model", "media", "*"}
TIMING	All values used as keys in the 'times' dictionary of the FuncExecDesc dictionary (6.1)
OPCODE	{"route", "switch"}
DEVMODEL	Union of SWITCHMODEL, ROUTERMODEL

DEVIMODEL	UNION OF SWITCHMODEL and ROUTERMODEL
METHOD	Every Func class defines identifiers for the different computational activities its members may engage in, depending on the specifics of the message arrival that triggered them. METHOD is the set of these, although in practice a reference to a methodcode on an edge requirements that the class of the Func which is the destination of the edge must define that methodcode.